

Licenciatura em Engenharia Informática

LEI-ISEP 5th Semester Integrative Project

PART III – Sprint 1 – User Stories

Version v1.0

This document defines the User Stories and acceptance criteria for Sprint 1 of the **LaPrizza** Integrative Project. The sprint focuses on establishing the project environment and engineering practices, modelling the core business domain, defining the architectural foundations of the information system, and developing the first iteration of the back-end application and its REST APIs.

2026/27

Contents

III.1 Preamble	1	BCK07 Pizza Component Type Management	8
III.2 Sprint 1	2	BCK08 Pizza Component Configuration	9
III.2.1 User Stories	2	BCK09 Stock Information Management	9
ENG Project Setup and Engineering Practices	2	BCK10 Predefined Pizza Management	10
ENG01 Project Repository and Development Environment	3	BCK11 Product Catalogue and Product Information	10
ENG02 Sprint Planning and Project Monitoring	3	ARC Architecture and Domain Modelling	11
ENG03 Self and Peer Evaluation	4	ARC01 Core Domain Analysis and Modelling	11
BCK Back-End	4	ARC02 System Architecture	12
BCK01 User and Authentication Management	5	ARC03 REST API	13
BCK02 Customer Management	6	ARC04 Persistence and Data Integrity	13
BCK03 Product Category Management	6	ARC05 Automated Testing	14
BCK04 Allergen Management	7	III.2.2 Sprint 1 User Story Summary	14
BCK05 Product Management	7	III.2.3 Sprint 1 Deliverable	15
BCK06 Pizza Size Management	8		

III.1 Preamble

The User Stories presented in this document define the work expected during **Sprint 1** of the fifth-semester Integrative Project.

They must be interpreted together with the information system description and general requirements established in *PART II – System Specification*. The general organization, working methodology, team rules, and assessment principles defined in *PART I – Context and Operation* also apply throughout this sprint.

Sprint 1 constitutes the first development iteration of the **LaPrizza** information system. Its purpose is not to develop the complete system, but to establish its technical and organizational foundations, analyse and model its core business domain, define the architectural foundations of the information system as a whole, and deliver the first coherent implementation of its back-end functionality.

According to the project calendar, Sprint 1 runs from **14 September 2026 to 25 October 2026**. During this sprint, the participating course units are **LAPR5** and **ARQSI**.

III.2 Sprint 1

The aim of Sprint 1 is threefold:

- 1** to set up the tools and infrastructure required to properly develop and manage the project, following appropriate software-engineering practices;
- 2** to design and develop the first iteration of the **LaPrizza** back-end application, exposing REST APIs for managing the principal business information required during this sprint; and
- 3** to analyse and model the core **LaPrizza** business domain and define an appropriate architecture for the information system as a whole, considering the functionality, components, and integrations expected throughout the project.

The resulting software must constitute an **integrated and demonstrable increment** of the system. The architectural, domain-modelling, and implementation decisions adopted during this sprint should establish appropriate foundations for the progressive development and integration of the additional functionality and technological areas introduced in subsequent sprints.

Sprint 1 – Implementation Scope

The domain analysis and architectural design performed during Sprint 1 are expected to consider a broader portion of the **LaPrizza** information system than the functionality implemented during this sprint.

The concepts identified in the domain model and the components considered in the system architecture therefore do not necessarily correspond to functionality that must already be implemented. The **functional implementation scope of Sprint 1 is established by the Back-End User Stories** defined in this document.

III.2.1 User Stories

ENG — Project Setup and Engineering Practices

The following User Stories establish the project organization, development environment, and engineering practices required to support the development of the **LaPrizza** Integrative Project. They provide a common foundation for collaborative development, project planning and monitoring, version control, and the systematic management and assessment of the work performed throughout the project. Although introduced during Sprint 1, these practices are expected to be consistently followed and maintained throughout subsequent development iterations.

ENG01 Project Repository and Development Environment

As a Project Manager, I want the team to set up the tools and infrastructure required to develop and manage the project, **so that** the team can work collaboratively and follow appropriate software-engineering practices.

Acceptance Criteria and Comments

- GitHub must be used as the Version Control System (VCS);
- the project repository must be created inside the appropriate DEI organization on GitHub;
- the repository name must follow the pattern `LEI-SEM5-PI-2026-27-XXX-YY`, where `XXX` is the team class (e.g., `3DC`) and `YY` is the team number (e.g., `04`);
- the repository structure must be suitable for accommodating the different applications, modules, technologies, and documentation that will be developed throughout the semester;
- an appropriate branching and integration strategy must be defined, documented, and consistently followed by the team;
- issue/task tracking and a project board must be adopted to support planning and monitoring of the team's work;
- the repository must contain the information required for a new developer to understand the project structure and prepare an appropriate development environment;
- relevant project decisions, documentation, and evidence must be maintained under version control whenever appropriate;
- at the end of each sprint, the repository must be tagged to identify the exact version of the project submitted for assessment. The tag must follow the naming convention `sprint-N-submission`, where `N` identifies the corresponding sprint (e.g., `sprint-1-submission`). Once the submission tag has been created and the corresponding deliverable has been submitted, the tag must not be deleted, recreated, or moved to a different commit;
- the repository archive (.zip) corresponding to the sprint submission tag must be downloaded from the associated GitHub tag page (🔗) and submitted through the corresponding sprint submission activity on Moodle;
- the submitted archive and the repository tag must identify the same version of the project. Changes committed after the Sprint 1 submission tag was created are not considered part of the submitted Sprint 1 deliverable.

ENG02 Sprint Planning and Project Monitoring

As a Project Manager, **I want** the team to organize and monitor the work required for Sprint 1, **so that** progress, responsibilities, and deviations from the sprint plan can be identified throughout the iteration.

Acceptance Criteria and Comments

- The Sprint 1 User Stories must be analysed and decomposed into appropriate tasks;
- tasks must identify their status and the team member or members responsible for their execution;
- dependencies between relevant tasks should be identified;
- the project board must reflect the actual state of the project and be regularly updated throughout the sprint;
- relevant implementation work must be associated with the corresponding tasks or User Stories whenever applicable;
- the team must maintain sufficient evidence to allow the development process and individual contributions to be analysed during Sprint 1 assessment.

ENG03 Self and Peer Evaluation

As a Project Manager, **I want** each team member to assess their own contribution and the contributions of the other team members at the end of the sprint, **so that** individual participation, collaboration, and contribution to the team's work can be considered during the assessment process.

Acceptance Criteria and Comments

- Each team member must complete the Sprint 1 self and peer evaluation individually;
- each team member must evaluate their own contribution and the contribution of every other member of the team;
- evaluations must consider the work and contributions made throughout the sprint, rather than only the final deliverable;
- evaluations should be consistent with the available evidence of individual participation and contribution to the project;
- each team member is individually responsible for submitting their evaluation through the mechanism defined for the sprint;
- the evaluation must be submitted within the deadline established for Sprint 1 assessment.

BCK — Back-End

The following User Stories establish the first set of business capabilities to be implemented by the **LaPrizza** back-end application.

The implementation must follow the domain concepts and business rules introduced in *PART II – System Specification* and the domain model developed during this sprint.

The resulting functionality must be persistently maintained and exposed through appropriate REST APIs, providing a coherent foundation for the functionality and integrations introduced in subsequent sprints.

BCK01 User and Authentication Management

As a LaPrizza Manager, I want the system to maintain users and roles and authenticate registered users, **so that** access to protected operations can be associated with an identified and authorized user.

Acceptance Criteria and Comments

- The system must support the registration and management of the users required to access protected system functionality;
- each user must have a unique identifier and appropriate authentication credentials;
- the system must support the roles required to distinguish the responsibilities and permitted operations of different users;
- registered users must be able to authenticate using their credentials;
- successful authentication must establish the identity of the user for subsequent protected operations;
- invalid authentication attempts must not provide access to protected functionality;
- authentication credentials must be handled and stored using appropriate security practices;
- the resulting authentication and role model must provide an appropriate foundation for the authorization requirements introduced throughout the project.

BCK02 Customer Management

As a LaPrizza Manager, I want to register and manage customers, **so that** the restaurant can maintain the information required to provide its services to them.

Acceptance Criteria and Comments

- The system must support the creation and update of customer records;
- each customer must have a unique identifier;
- customer information must include, at least, the customer's name and appropriate contact information;
- data used to uniquely identify or contact customers must be appropriately validated;
- customers must be searchable using relevant identifying information;
- the solution must allow relevant dietary restrictions associated with a customer to be represented;
- customer information that is referenced by other business information must not be removed in a manner that compromises the consistency of that information.

BCK03 Product Category Management

As a LaPrizza Manager, I want to maintain product categories, **so that** the products managed by the restaurant can be appropriately classified.

Acceptance Criteria and Comments

- The system must maintain a catalogue of Product Categories;
- each Product Category must have a unique identifier and a descriptive name;
- products must be associable with an appropriate Product Category;
- Product Categories must be searchable using relevant identifying information;
- a Product Category referenced by existing products must not be removed in a manner that compromises the consistency of the corresponding business information.

BCK04 Allergen Management

As a LaPrizza Manager, I want to maintain the allergens recognized by the restaurant, **so that** products can be associated with appropriate allergen information.

Acceptance Criteria and Comments

- The system must maintain a catalogue of allergens;
- each allergen must have a unique identifier and a descriptive name;
- allergens must be available for association with products;
- a product may be associated with one or more allergens;
- allergens must be searchable using relevant identifying information;
- an allergen referenced by existing business information must not be removed in a manner that compromises the consistency of that information.

BCK05 Product Management

As a LaPrizza Manager, I want to register and manage products, **so that** the restaurant can maintain the items relevant to its business operation.

Acceptance Criteria and Comments

- The system must support the creation and update of products;
- each product must have a unique identifier, a name, and a description;
- a product must be associable with an appropriate Product Category;
- a product may be designated as sellable, subject to stock control, suitable for use as a Pizza Component, or any applicable combination of these characteristics, according to the business rules defined in the System Specification;
- a product designated as a Pizza Component must be associated with exactly one Pizza Component Type;
- sellable products must provide the commercial information required to make them available for sale;
- products may be associated with one or more allergens;
- products must be searchable and filterable using relevant information;
- the system must allow products that should no longer be actively used to be distinguished without compromising historically relevant information;
- changes to product information must preserve the consistency of existing relationships with other business concepts.

BCK06 Pizza Size Management

As a LaPrizza Manager, I want to maintain the pizza sizes offered by the restaurant, **so that** pizza configurations and size-dependent information can refer to a consistent set of available sizes.

Acceptance Criteria and Comments

- The system must maintain the Pizza Sizes recognized by **LaPrizza**;
- each Pizza Size must have a unique identifier and a descriptive name;
- Pizza Sizes must be available for use in pizza-related configurations;
- Pizza Sizes must be searchable using relevant identifying information;
- the system must allow the set of available Pizza Sizes to evolve without compromising existing business information;
- a Pizza Size referenced by existing business information must not be removed in a manner that compromises its consistency.

BCK07 Pizza Component Type Management

As a LaPrizza Manager, I want to maintain Pizza Component Types, **so that** products suitable for use in pizzas can be classified according to the role they play in a pizza configuration.

Acceptance Criteria and Comments

- The system must maintain the Pizza Component Types recognized by **LaPrizza**;
- each Pizza Component Type must define its relative assembly order within a pizza configuration, and two Pizza Component Types must not have the same assembly order;
- each Pizza Component Type must define the minimum and maximum number of components of that type that may participate in a pizza configuration;
- the maximum number of components may be left unbounded;
- the set of Pizza Component Types and their properties must be configurable and must not be unnecessarily fixed in the application source code;
- the initial configuration must include, at least, the following Pizza Component Types:
 - ◆ Base Dough: minimum 1, maximum 1, assembly order 1;
 - ◆ Base Sauce: minimum 0, maximum 1, assembly order 2;
 - ◆ Cheese: minimum 0, maximum 1, assembly order 3;
 - ◆ Topping: minimum 0, unbounded maximum, assembly order 4.
- Pizza Component Types must be searchable using relevant identifying information;
- a Pizza Component Type referenced by existing business information must not be removed in a manner that compromises its consistency.

BCK08 Pizza Component Configuration

As a LaPrizza Manager, I want to configure the size-dependent characteristics of products used as pizza components, **so that** the system can determine the information applicable when a component is used in a pizza of a particular size.

Acceptance Criteria and Comments

- Only products designated for use as Pizza Components may have Component Size Configurations;
- a Component Size Configuration must associate a Pizza Component with a Pizza Size;
- for a given Pizza Component and Pizza Size, at most one applicable Component Size Configuration may exist;
- a Component Size Configuration must define the price applicable to the component when used in a pizza of the corresponding size;
- when the corresponding product is subject to stock control, the Component Size Configuration must allow the number of abstract stock units consumed for that Pizza Size to be represented;
- prices and stock-consumption values must satisfy the corresponding business validation rules;
- the system must allow the size-dependent configurations of a Pizza Component to be consulted;
- changes to Component Size Configurations must preserve the consistency of existing business information.

BCK09 Stock Information Management

As a LaPrizza Manager, I want to maintain stock information for products subject to stock control, **so that** the restaurant knows the quantity currently available for each applicable product.

Acceptance Criteria and Comments

- Stock information must only be maintained for products designated as subject to stock control;
- each stock-controlled product must have an identifiable current stock quantity;
- stock quantities must be represented using the abstract stock-unit model defined in the System Specification;
- stock quantities must be represented as non-negative integer numbers of abstract stock units;
- the system must allow the current stock level of an individual stock-controlled product to be consulted;

- stock information must be searchable using relevant product information;
- the system must provide operations for increasing or decreasing the current stock quantity by a specified number of abstract stock units;
- a stock reduction must not result in a negative stock quantity;
- operations that change stock quantities must validate the requested change and preserve the consistency of the resulting stock information;
- the stock model must provide an appropriate foundation for the ordering and operational functionality introduced in subsequent sprints.

BCK10 Predefined Pizza Management

As a LaPrizza Manager, I want to define and manage predefined pizzas, **so that** the restaurant can maintain the standard pizza configurations offered to its customers.

Acceptance Criteria and Comments

- A Predefined Pizza must be represented by an appropriate sellable product;
- a Predefined Pizza must define a default pizza configuration composed of products designated as Pizza Components;
- only active products currently designated as Pizza Components may be included in a default pizza configuration;
- the same Pizza Component must not occur more than once in the same default pizza configuration;
- the default configuration must satisfy the minimum and maximum cardinality rules defined for each Pizza Component Type;
- a Predefined Pizza may only be made available in Pizza Sizes for which all Pizza Components in its default configuration have a corresponding Component Size Configuration;
- the system must allow the default configuration of a Predefined Pizza to be consulted;
- the information maintained for a Predefined Pizza must provide an appropriate basis for the pizza customization functionality introduced in subsequent sprints.

BCK11 Product Catalogue and Product Information

As a Customer, I want to consult the products offered by **LaPrizza**, **so that** I can understand the available options, their pricing, and other relevant product information.

Acceptance Criteria and Comments

- The system must provide access to the products currently offered for sale by **LaPrizza**;
- the catalogue must provide relevant information about each sellable product, including,

where applicable, its name, description, category, pricing information, and allergen information;

- users must be able to consult the details of an individual sellable product;
- for a Predefined Pizza, the catalogue must provide its default configuration and the Pizza Sizes in which it is available;
- the information presented in the catalogue must be consistent with the corresponding business information maintained by the system.

ARC — Architecture and Domain Modelling

The following User Stories establish the domain-modelling and architectural foundations for the **LaPrizza** information system. The domain analysis and system architecture have a broader scope than the functionality implemented during Sprint 1, while the remaining User Stories define architectural and quality requirements applicable to the Sprint 1 back-end increment.

The resulting models and architectural decisions should provide a coherent, maintainable, and extensible foundation for the progressive development of the system and for the integration of the different applications and modules introduced in subsequent sprints.

Although the domain analysis and system architecture must consider the wider system, the functionality required to be implemented during Sprint 1 is limited to that defined by the Back-End User Stories of this sprint.

ARC01 Core Domain Analysis and Modelling

As a Development Team, **we want** to analyse and model the core **LaPrizza** business domain, **so that** the software design reflects the relevant business concepts, rules, and relationships and provides a coherent foundation for subsequent development.

Acceptance Criteria and Comments

- The team must analyse the business concepts and relationships introduced in *PART II – System Specification*;
- the resulting domain model must identify the principal domain concepts, their responsibilities, relevant relationships, and important business rules;
- the domain analysis must consider, at least, the principal concepts related to customers, dietary restrictions, products, product categories, allergens, pizza sizes, pizza component types, component size configurations, stock information, predefined pizzas, pizza configurations, orders, order items, preparation activities, and kitchen resources;
- relevant relationships and constraints between these concepts must be explicitly represented and justified;
- the model should distinguish domain concepts from implementation, persistence, presentation, and infrastructure concerns;

- appropriate modelling artefacts must be produced and maintained as part of the project's technical documentation;
- important modelling decisions and assumptions must be documented whenever they are not evident from the corresponding artefacts;
- the implemented Sprint 1 business model must be consistent with the corresponding portion of the documented domain model;
- the model must provide an appropriate foundation for the functionality planned for subsequent sprints and should avoid design decisions that unnecessarily constrain its evolution;
- not all concepts identified in the domain model are required to be implemented during Sprint 1. The functional scope of the back-end increment is established by the Back-End User Stories defined for this sprint.

ARC02 System Architecture

As a Development Team, **we want** to define an appropriate architecture for the **LaPrizza** information system, **so that** its different applications, modules, and responsibilities can evolve and be progressively integrated throughout the project.

Acceptance Criteria and Comments

- The architecture must identify the principal applications, modules, and external systems expected to participate in the **LaPrizza** information system;
- the responsibilities of the principal system components and the relevant interactions between them must be identified;
- the architecture must consider the progressive integration of the back-end application, Web Application, 3D Visualization, Artificial Intelligence, and other system components introduced throughout the project;
- the architecture of the back-end application implemented during Sprint 1 must clearly identify its main components or layers and their responsibilities;
- responsibilities and concerns must be appropriately separated;
- domain and business rules must not be unnecessarily coupled to presentation, persistence, or infrastructure mechanisms;
- dependencies between the major components of the solution must be explicitly identified and must follow the architectural approach adopted by the team;
- the architecture must support the progressive addition and integration of functionality during subsequent sprints;
- relevant architectural decisions and their rationale must be documented;
- appropriate architectural views or diagrams must be included in the technical documentation;
- the implemented Sprint 1 components must be consistent with the corresponding portion of the documented architecture.

ARC03 REST API

As a Development Team, **we want** the Sprint 1 business functionality to be exposed through a REST API, **so that** different clients and system modules can interact with the **LaPrizza** information system through well-defined interfaces.

Acceptance Criteria and Comments

- The business functionality implemented during Sprint 1 must be exposed through appropriate REST API endpoints;
- API resources and operations must follow consistent design and naming conventions;
- appropriate HTTP methods must be used according to the semantics of the corresponding operations;
- appropriate HTTP response status codes must be returned;
- input data must be appropriately validated before the corresponding business operations are executed;
- API operations must preserve the business rules and invariants established by the domain model;
- error responses must provide sufficient information for clients to identify the nature of the problem without exposing inappropriate implementation details;
- API contracts must be documented using an appropriate machine-readable API description mechanism;
- the documented API must remain consistent with its implementation;
- the API design must consider its future consumption by the Web Application, 3D Visualization, Artificial Intelligence, and other system components.

ARC04 Persistence and Data Integrity

As a Development Team, **we want** the business information managed by the back-end application to be persistently and consistently stored, **so that** the system preserves valid business information across its operations.

Acceptance Criteria and Comments

- The persistence model must support the business concepts and relationships required by the Sprint 1 implementation;
- business identifiers and relevant uniqueness constraints must be appropriately enforced;
- relationships between persisted business information must preserve referential and business consistency;
- persistence mechanisms must not allow the business invariants established by the implemented domain model to be bypassed;

- persistence concerns should remain appropriately separated from core domain and business logic;
- the solution must provide an appropriate mechanism for creating and evolving the persistence structures required by the application;
- the persistence solution must allow the application to be initialized in a reproducible manner;
- relevant design decisions concerning persistence, mapping, and data integrity must be documented.

ARC05 Automated Testing

As a Development Team, **we want** the implemented business functionality to be automatically tested, **so that** its behaviour can be verified and regressions can be identified as the system evolves.

Acceptance Criteria and Comments

- Relevant domain and business rules implemented during Sprint 1 must be covered by automated tests;
- relevant application and API behaviour must be automatically tested where appropriate;
- tests must include representative valid and invalid scenarios;
- tests must verify important business invariants and validation rules;
- the test suite must be repeatable and capable of being executed by any team member in an appropriately configured development environment;
- test code must be maintained under version control together with the corresponding production code;
- the team must maintain sufficient testing evidence to support the assessment of the implemented functionality and its quality.

III.2.2 Sprint 1 User Story Summary

Table 1 provides an overview of the User Stories defined for Sprint 1, organized according to their corresponding project areas.

Table 1: Summary of Sprint 1 User Stories.

Area	US	User Story
ENG	ENG01	Project Repository and Development Environment
ENG	ENG02	Sprint Planning and Project Monitoring

Continued on next page

Table 1 – continued from previous page

Area	US	User Story
ENG	ENG03	Self and Peer Evaluation
BCK	BCK01	User and Authentication Management
BCK	BCK02	Customer Management
BCK	BCK03	Product Category Management
BCK	BCK04	Allergen Management
BCK	BCK05	Product Management
BCK	BCK06	Pizza Size Management
BCK	BCK07	Pizza Component Type Management
BCK	BCK08	Pizza Component Configuration
BCK	BCK09	Stock Information Management
BCK	BCK10	Predefined Pizza Management
BCK	BCK11	Product Catalogue and Product Information
ARC	ARC01	Core Domain Analysis and Modelling
ARC	ARC02	System Architecture
ARC	ARC03	REST API
ARC	ARC04	Persistence and Data Integrity
ARC	ARC05	Automated Testing

III.2.3 Sprint 1 Deliverable

Collectively, the User Stories defined above establish the expected scope of the Sprint 1 work and deliverable.

At the end of Sprint 1, the team is expected to demonstrate a **working and integrated first iteration of the LaPrizza information system**.

At a minimum, the Sprint 1 deliverable and associated assessment evidence should demonstrate:

- an appropriately configured project repository and project-management environment;
- analysis and modelling of the core **LaPrizza** business domain, including the principal concepts required during subsequent project iterations;
- a coherent architecture for the **LaPrizza** information system as a whole, including an appropriate architecture for the back-end application implemented during Sprint 1;
- management and authentication of users, including the roles required to support access to protected system functionality;

- persistent management of the principal business entities required by the Sprint 1 Back-End User Stories;
- management of customers, Product Categories, allergens, products, Pizza Sizes, and Pizza Component Types;
- management of size-dependent Pizza Component configurations and their applicable pricing and stock-consumption information;
- basic management and consultation of stock information using the abstract stock-unit model;
- definition and consultation of Predefined Pizzas and their default configurations;
- consultation of sellable product information, including relevant product and allergen information and, for Predefined Pizzas, their default configurations and available Pizza Sizes;
- appropriate REST APIs supporting the implemented functionality;
- a persistence solution consistent with the implemented domain model and business rules;
- automated tests covering relevant business rules and implemented functionality;
- technical and project documentation consistent with the implemented solution;
- completion and submission of the individual Sprint 1 self and peer evaluations.

Sprint 1 Completion

A User Story should only be considered complete when all of its acceptance criteria have been satisfied and the corresponding results and evidence are available for assessment.

For User Stories involving software functionality, that functionality must be **implemented, integrated, tested, documented where appropriate, and demonstrable** as part of the common project solution.

The presence of a concept in the domain model or a component in the system architecture does not, by itself, imply that the corresponding functionality must be implemented during Sprint 1. The functional implementation scope is determined by the Back-End User Stories assigned to this sprint.

For User Stories involving software functionality, partial implementations, isolated prototypes, or functionality that cannot be integrated and demonstrated as part of the **LaPrizza** system should not be considered complete.